LAB: Stepper Motor

Date: 2024-10-23

Author/Partner: HWANG MooYeol

Github: repository link

Demo Video: Youtube link

PDF version:LAB Stepper Motor PDF

Introduction

In this lab, we will learn how to drive a stepper motor with digital output of GPIOs of MCU. You will use a FSM to design the algorithm for stepper motor control.

You must submit

- LAB Report (*.pdf & *.md)
- Zip source files(main*.c, ecRCC.h, ecGPIO.h, ecSysTick.c etc...).
    - Only the source files. Do not submit project files

Requirement

Hardware

- MCU
    - NUCLEO-F411RE
- Actuator/Sensor/Others:
    - 3Stepper Motor 28BYJ-48
    - Motor Driver A4988 (tutorial)
    - Motor Driver ULN2003 (lab)
    - breadboard

Software

- Keil uVision, CMSIS, EC_HAL library

Problem : Stepper Motor with 4-input sequence

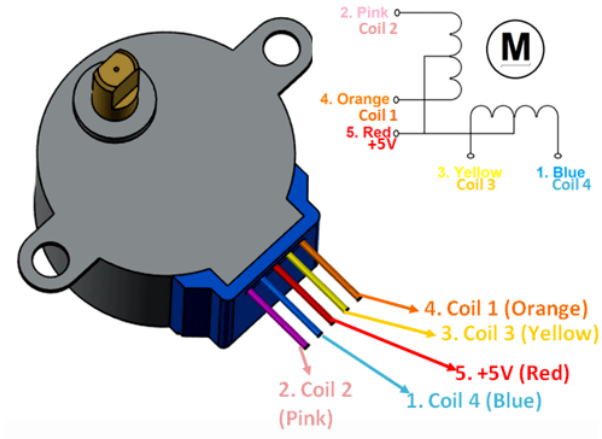
For the lab, we are going to use another stepper motor driver of **ULN2003 motor driver** See here for ULN2003 spec sheet)

Here, you have to give 4-input pulses in sequence.

Hardware Connection

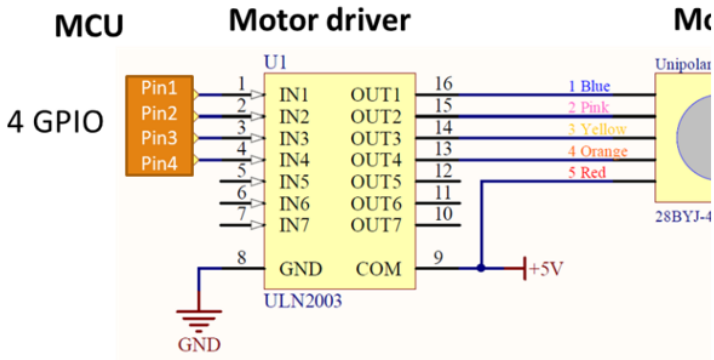
Read specification sheet of the motor and the motor driver for wiring and min/max input voltage/current.

Unipolar Stepper Motor 28BYJ-48



- Rated Voltage: 5V DC
- Number of Phases: 4
- Stride Angle: 5.625°/64
- Gear ratio: 1/32
- Pull in torque: 300 gf.cm
- Coil: Unipolar 5 lead coil

MCU connection wiring



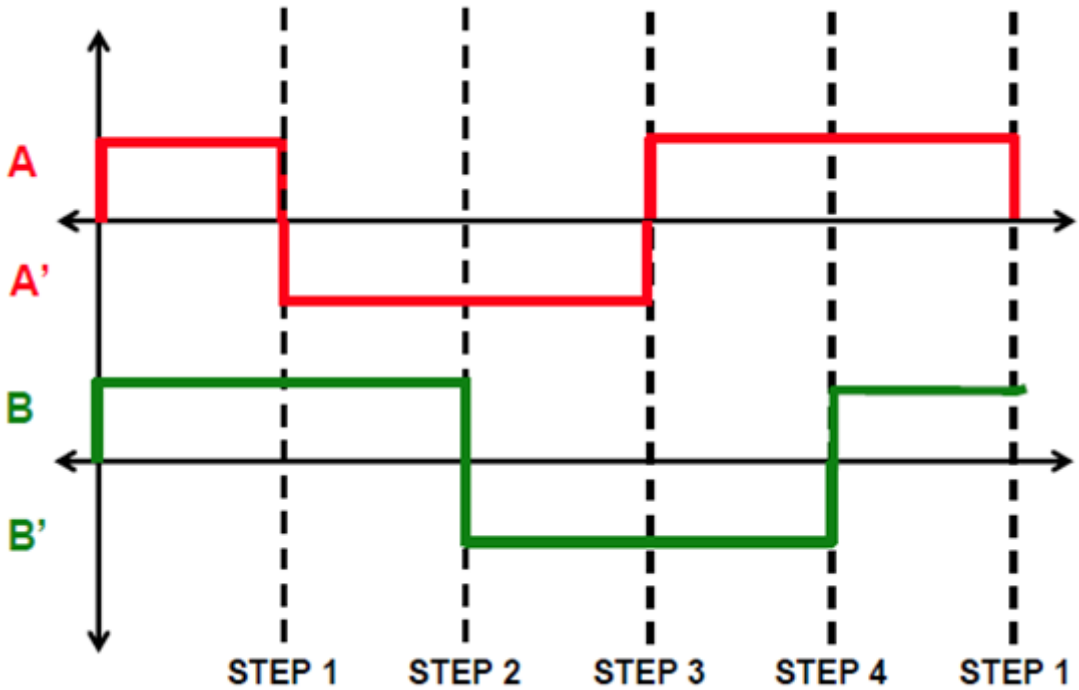
Motor driver (ULN2003)

Stepper Motor Sequence

We will use unipolar stepper motor for this lab

Fill in the blanks of each output data depending on the below sequence.

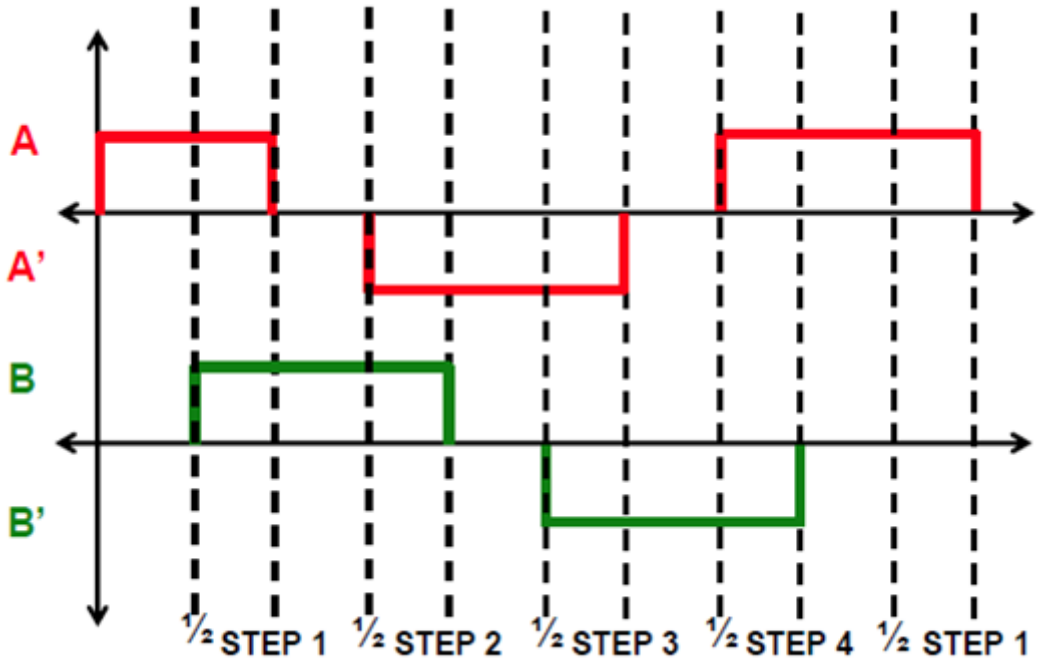
Full-stepping sequence



Phase	Port_Pin	Sequence			
		1	2	3	4
A	PB_10	H	L	L	H
B	PB_4	H	H	L	L
A′	PB_5	L	H	H	L
B′	PB_3	L	L	H	H

Full-stepping Sequence

Half-stepping sequence



Phase	Port_Pin	Sequence							
		1	2	3	4	5	6	7	8
A	PB_10	H	H	L	L	L	L	L	H
B	PB_4	L	H	H	H	L	L	L	L
A′	PB_5	L	L	L	H	H	H	L	L
B′	PB_3	L	L	L	L	L	H	H	H

Half-stepping Sequence

Finite State Machine

Draw a State Table for Full-Step Sequence. You can choose either Moore FSM or Mealy.

- Full-Stepping Sequence (Moore)

State	Next State		Output
	DIR = 0	DIR = 1	(A B A′ B′)
S0	S3	S1	H H L L
S1	S0	S2	L H H L
S2	S1	S3	L L H H
S3	S2	S0	H L L H

- Half-Stepping Sequence (Moore)

State	Next State		Output
	DIR = 0	DIR = 1	(A B A′ B′)
S0	S7	S1	H L L L
S1	S0	S2	H H L L
S2	S1	S3	L H L L
S3	S2	S4	L H H L
S4	S3	S5	L L H L
S5	S4	S6	L L H H
S6	S5	S7	L L L H
S7	S6	S0	H L L H

You have to program the stepping sequence using the state table. You can define the states using structures.

```
//State number
#define S0 0
#define S1 1
#define S2 2
#define S3 3
#define S4 4
#define S5 5
#define S6 6
#define S7 7

//FULL stepping sequence  - FSM (Moore)
typedef struct {
    uint32_t next[2];
        uint8_t out;
} State_full_t;

State_full_t FSM_full[4] = {
    {{S3,S1},0b1100},    //dir=0,1  output ABA'B'
    {{S0,S2},0b0110},
    {{S1,S3},0b0011},
    {{S2,S0},0b1001}
};

//HALF stepping sequence
typedef struct {
  uint32_t next[2];
    uint8_t out;
} State_half_t;
```

```
State_half_t FSM_half[8] = {
    {{S7,S1},0b1000},   //dir=0,1  output ABA'B'
    {{S0,S2},0b1100},
    {{S1,S3},0b0100},
    {{S2,S4},0b0110},
    {{S3,S5},0b0010},
    {{S4,S6},0b0011},
    {{S5,S7},0b0001},
    {{S6,S0},0b1001}
};
```

Create HAL library

Declare and define the following functions in your library.

ecStepper.h

```
// Initialize with 4 pins
// ( A, B,  AN,  BN)
void Stepper_init(PinName_t A, PinName_t B,  PinName_t AN, PinName_t BN);

// whatSpeed [rev/min]
void Stepper_setSpeed(long whatSpeed);

// Run for n Steps
void Stepper_step(uint32_t steps, uint32_t direction, uint32_t mode);

// Immediate Stop.
void Stepper_stop(void);
```

ecStepper.c

```
#include "stm32f4xx.h"
#include "ecStepper2.h"

//State number
#define S0 0
#define S1 1
#define S2 2
#define S3 3
#define S4 4
#define S5 5
#define S6 6
#define S7 7


// Stepper Motor function
uint32_t direction = 1;
double step_delay = 1.0;
double step_per_rev = 64.0*32.0;


// Stepper Motor variable
volatile Stepper_t myStepper;
```

```
//FULL stepping sequence  - FSM
typedef struct {
    uint32_t next[2];
        uint8_t out;
} State_full_t;

State_full_t FSM_full[4] = {    // 1010 , 0110 , 0101 , 1001
    {{S3,S1},0b1100},       // ABA'B'
    {{S0,S2},0b0110},
    {{S1,S3},0b0011},
    {{S2,S0},0b1001}
};

//HALF stepping sequence
typedef struct {
  uint32_t next[2];
    uint8_t out;
} State_half_t;

State_half_t FSM_half[8] = {    // 1000 , 1010 , 0010 , 0110 , 0100 , 0101,
0001, 1001
    {{S7,S1},0b1000},
     {{S0,S2},0b1100},
     {{S1,S3},0b0100},
     {{S2,S4},0b0110},
     {{S3,S5},0b0010},
     {{S4,S6},0b0011},
     {{S5,S7},0b0001},
     {{S6,S0},0b1001}
};



void Stepper_init(PinName_t pinName1, PinName_t pinName2, PinName_t pinName3,
PinName_t pinName4){

    //  GPIO Digital Out Initiation
    myStepper.pin1 = pinName1;
    myStepper.pin2 = pinName2;
    myStepper.pin3 = pinName3;
    myStepper.pin4 = pinName4;

    GPIO_init(pinName1, OUTPUT);
    GPIO_init(pinName2, OUTPUT);
    GPIO_init(pinName3, OUTPUT);
    GPIO_init(pinName4, OUTPUT);

    //  GPIO Digital Out Initiation
    // No pull-up Pull-down , Push-Pull, Fast
    // Pin1 ~ Pin4

    //pull-up, pull-down
    GPIO_pupd(pinName1, NO_PUPD);
    GPIO_pupd(pinName2, NO_PUPD);
    GPIO_pupd(pinName3, NO_PUPD);
    GPIO_pupd(pinName4, NO_PUPD);
    // type
```

```
    GPIO_otype(pinName1,PUSH_PULL);
    GPIO_otype(pinName2,PUSH_PULL);
    GPIO_otype(pinName3,PUSH_PULL);
    GPIO_otype(pinName4,PUSH_PULL);

    // speed
    GPIO_ospeed(pinName1, FAST_SPEED);
    GPIO_ospeed(pinName2, FAST_SPEED);
    GPIO_ospeed(pinName3, FAST_SPEED);
    GPIO_ospeed(pinName4, FAST_SPEED);
}


void Stepper_pinOut (uint32_t state, uint32_t mode){
    if (mode == FULL){         // FULL mode
        GPIO_write(myStepper.pin1, (FSM_full[state].out & 0b1000) >> 3);
        // Repeat for pin2~pin4
        GPIO_write(myStepper.pin2, (FSM_full[state].out & 0b0100) >> 2);
        GPIO_write(myStepper.pin3, (FSM_full[state].out & 0b0010) >> 1);
        GPIO_write(myStepper.pin4, (FSM_full[state].out & 0b0001) >> 0);
    }
    else if (mode == HALF){    // HALF mode
        GPIO_write(myStepper.pin1, (FSM_half[state].out & 0b1000) >> 3);
        GPIO_write(myStepper.pin2, (FSM_half[state].out & 0b0100) >> 2);
        GPIO_write(myStepper.pin3, (FSM_half[state].out & 0b0010) >> 1);
        GPIO_write(myStepper.pin4, (FSM_half[state].out & 0b0001) >> 0);
    }
}


void Stepper_setSpeed (double whatSpeed){      // rpm [rev/min]
        step_delay = 60000.0/(whatSpeed*step_per_rev);   // Convert rpm to
[msec/step] delay
}


void Stepper_step(uint32_t steps, uint32_t direction, uint32_t mode){
     uint32_t state = 0;
     myStepper._step_num = steps;

     for(; myStepper._step_num > 0; myStepper._step_num--){ // run for step size
       if (mode == FULL) {
            delay_ms(step_delay);
            state = FSM_full[state].next[direction];     // state = next state
         }
        else if (mode == HALF) {
            delay_ms(step_delay/2);
            state = FSM_half[state].next[direction];    // state = next state
        }
        Stepper_pinOut(state, mode);
    }
}


void Stepper_stop (void){
        myStepper._step_num = 0;
    // All pins(A,AN,B,BN) set as DigitalOut '0'
    GPIO_write(myStepper.pin1, 0);
```

```
    GPIO_write(myStepper.pin2, 0);
    GPIO_write(myStepper.pin3, 0);
    GPIO_write(myStepper.pin4, 0);
}
```

> Note that these are blocking stepper controllers. While the stepper is running, the MCU cannot process other polling commands. If you can, modify it to be the non-blocking controller.

MCU board initially operates within the main function. Unless there is an EXTI or Timer Interrupt event, the code inside the while loop doesn't execute. Therefore, the Stepper_Step function runs within the while loop, and with each occurrence of a SysTick interrupt, the msTicks variable used in the delay_ms() function is brought into the main code. It is then used to count, which triggers the next state.

Procedure

1. Connect the MCU to the motor driver and the stepper motor.
2. Find out the number of steps required to rotate 1 revolution using Full-steppping.

   The Stride Angle and Gear ratio is given by following: 5.625[deg] / 64 , 1/32

   Therefore, for 360[deg] (1 revolution) we need 64*32 = 2048 steps when Full stepping. For Half stepping we need twice steps then full stepping. Which is 4096 steps.
3. Then, rotate the stepper motor 10 revolutions with 2 rpm. Measure if the motor rotates one revolution per second.

   Full dir=0 : 10lap / 2rpm = 4:41 => 1lap = 28.1sec

   Full dir=1 : 10lap / 2rpm = 4:41 => 1lap = 28.1sec

   Half dir=0 : 10lap / 2rpm = 4:41 => 1lap = 28.1sec

   Half dir=1 : 10lap / 2rpm = 4:41 => 1lap = 28.1sec
4. Repeat the above process in the opposite direction.
5. Increase and decrease the speed of the motor as fast as it can rotate to find the maximum and minimum speed of the motor.

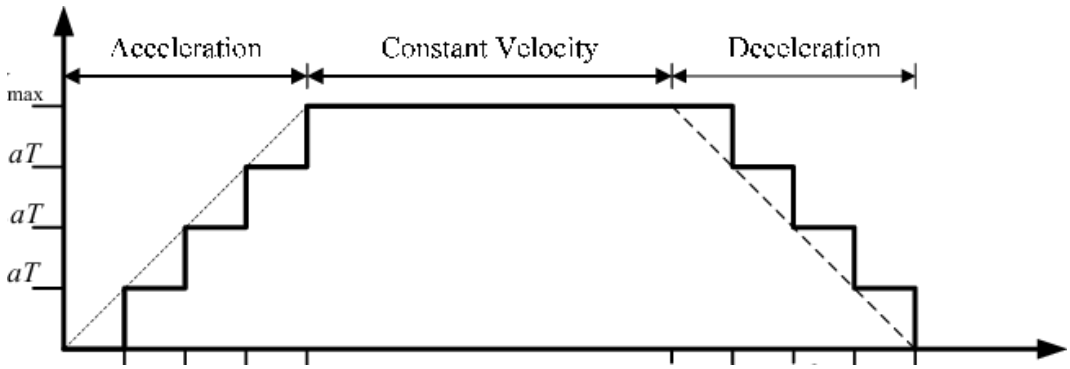
   Max = 14.64 rpm, Min < 0.01 rpm
6. Apply the half-stepping and repeat the above.

Configuration

Digital Out	SysTick
PB10, PB4, PB5, PB3 / NO Pull-up Pull-down / Push-Pull / Fast	delay()

Discussion

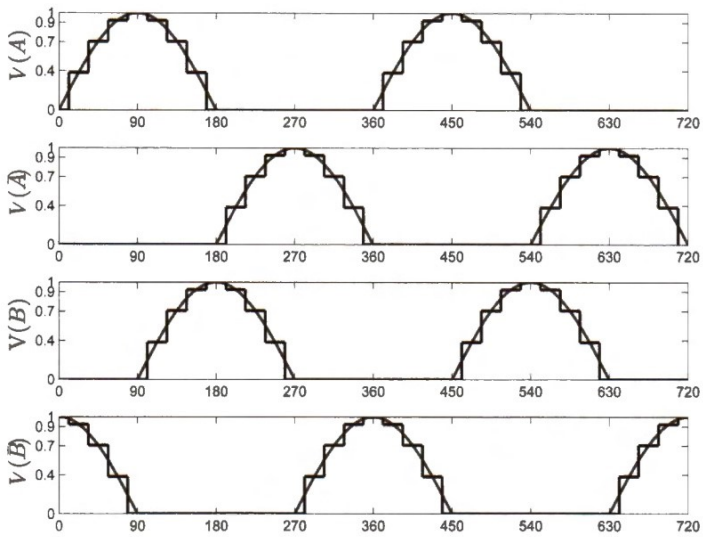
1. Find out the trapezoid-shape velocity profile for a stepper motor. When is this profile necessary?



Trapezoidal profile is necessary when need precise control of the motor's movement, especially when it's essential to minimize overshoot and ensure accurate positioning.

In Stepper motors, which move in discrete steps, can be more smoother at acceleration and deceleration. And also can minimize vibrations. Using feed-forward gain

2. How would you change the code more efficiently for micro-stepping control? You don't have to code this but need to explain your strategy.

> **We can use the PWM to perform micro-stepping, which rotates a fraction of a full step angle in one excitation, such as 1/4, 1/8, 1/16, or 1/32 of a full step.**
>
> **We can use PWM to generate a fast binary signal with an appropriately chosen duty cycle to adjust amplitude of the voltage across each winding dynamically.**
>
> **Following figure shows the voltage signal of each phase of unipolar stepper motors if 1/4 micro-strpping is used:**
>
> 
>
>
> **Figure 16-13. 1/4 sine-cosine micro-stepping for four-phase unipolar stepper motors**
>
> **Software should set the compare and capture register of the timer to the each micro-stepping ratio.**

Code

My code is here: [repository link]

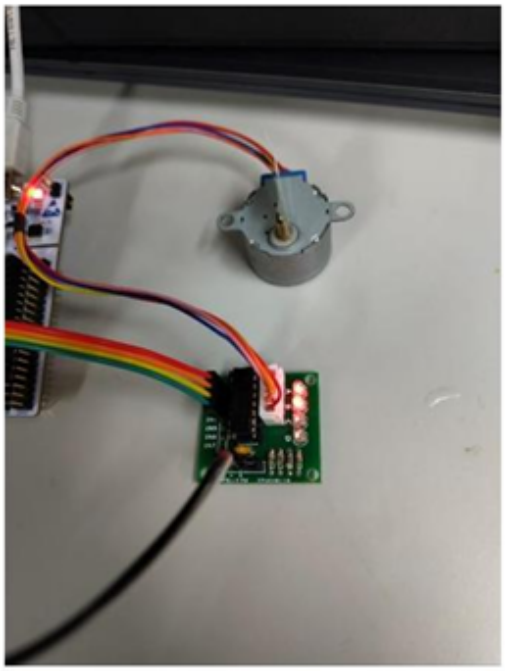
Code : Stepper Motor

```c
#include "stm32f411xe.h"


#include "ecGPIO2_student.h"
#include "ecRCC2.h"
#include "ecEXTI2.h"
#include "ecSysTick2.h"
#include "ecStepper2.h"

void setup(void);

int main(void) {
    // Initialiization -------------------------------------------------------
    setup();

    Stepper_step(2048, 0, FULL);  // (Step : Full: 2048 ,Half :4096 = 1 lap,
Direction : 0 or 1, Mode : FULL or HALF)

    // Inifinite Loop ---------------------------------------------------------
    while(1){;}


}

// Initialiization
void setup(void){

    RCC_PLL_init();                                 // System Clock = 84MHz
    SysTick_init();                                 // Systick init

    EXTI_init(BUTTON_PIN, FALL,0);           // External Interrupt Setting
    GPIO_init(BUTTON_PIN, INPUT);           // GPIOC pin13 initialization

    Stepper_init(PB_10,PB_4,PB_5,PB_3); // Stepper GPIO pin initialization
    Stepper_setSpeed(1);                            //  set stepper motor speed
  rpm
}

void EXTI15_10_IRQHandler(void) {
    if (is_pending_EXTI(BUTTON_PIN)) {
        Stepper_stop();
        delay_ms(1);
        clear_pending_EXTI(BUTTON_PIN); // cleared by writing '1'
    }
}
```

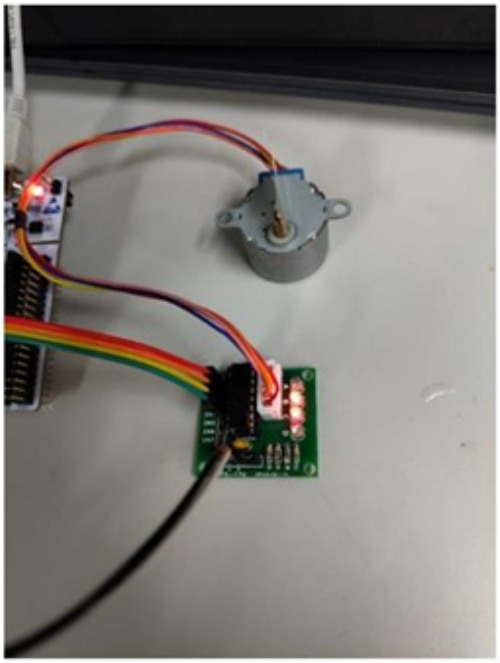
Results

Experiment images and results

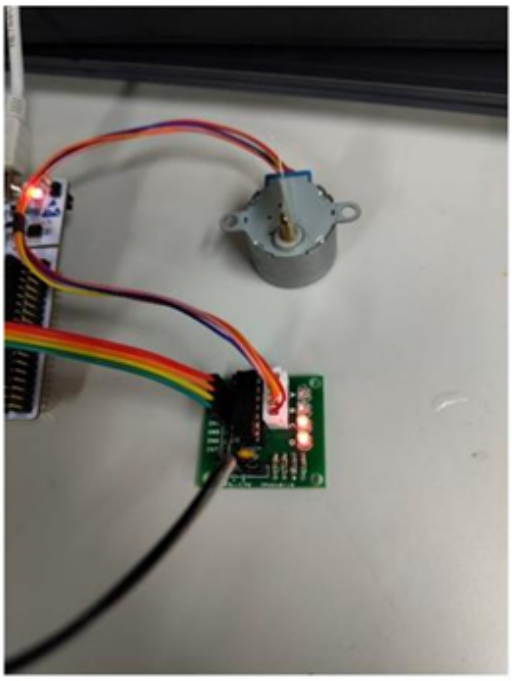
> **FULL STEPPING**



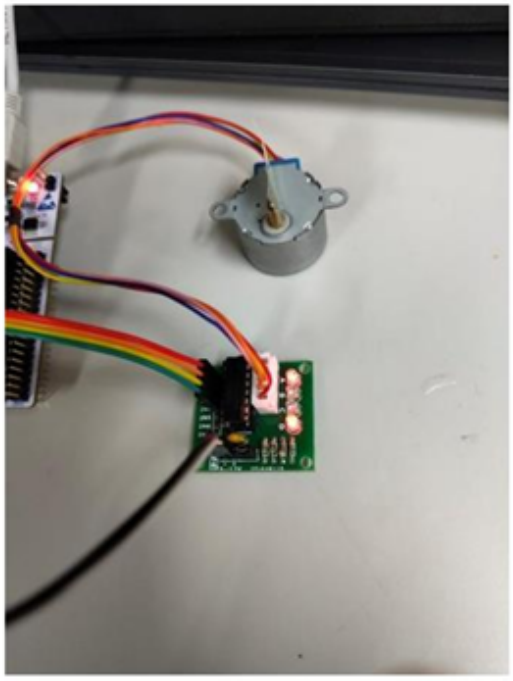
<S0>



<S1>



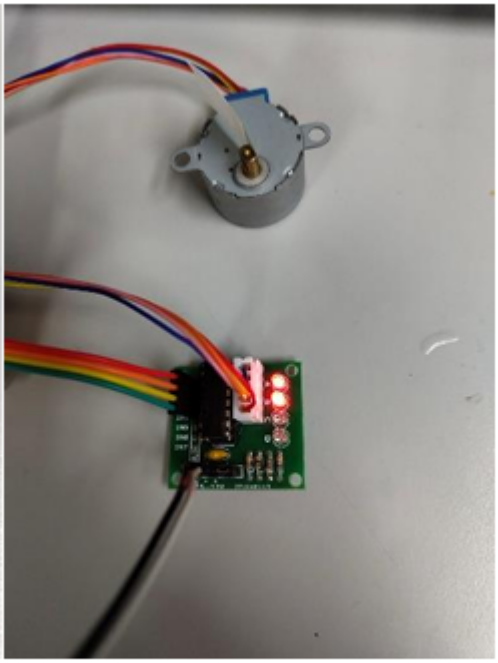
<S2>



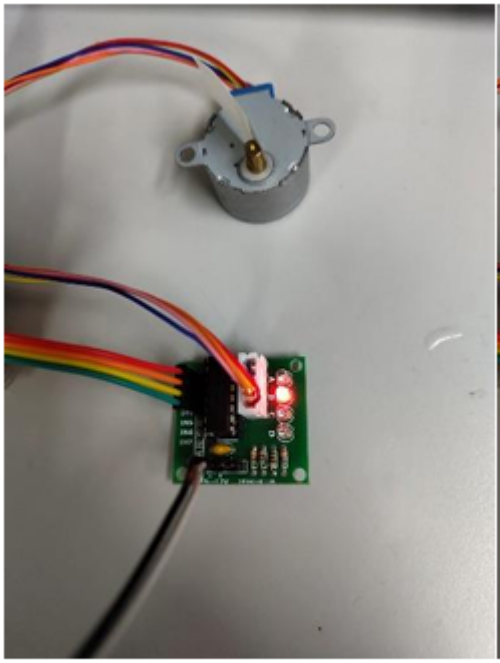
<S3>

HALF STEPPING

<S0>

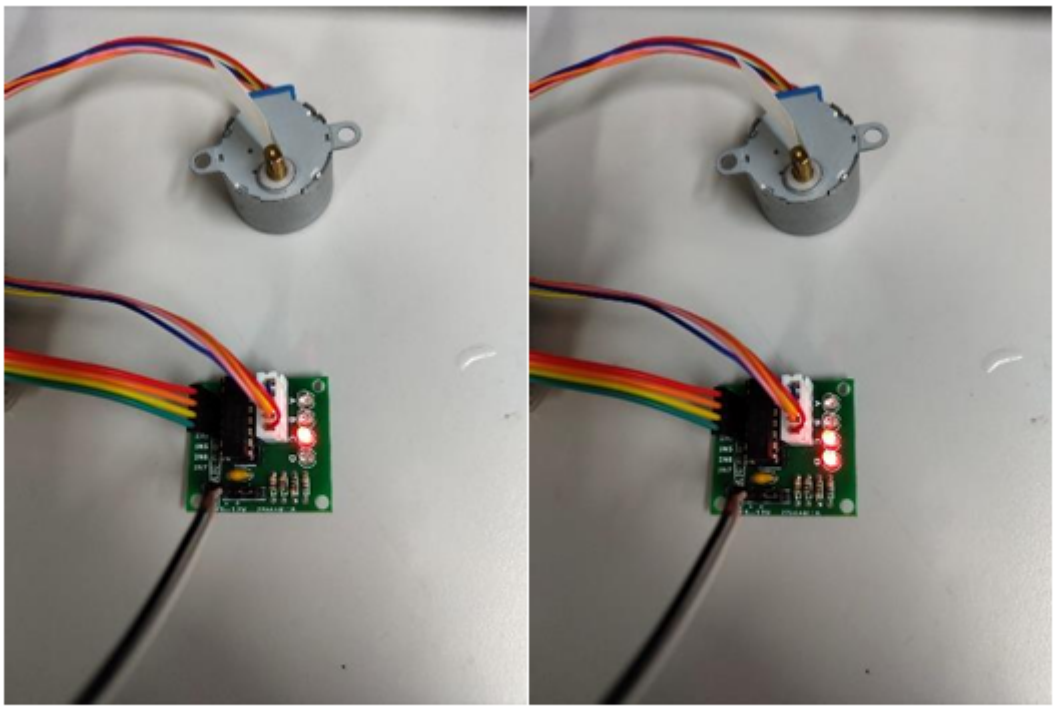


<S1>

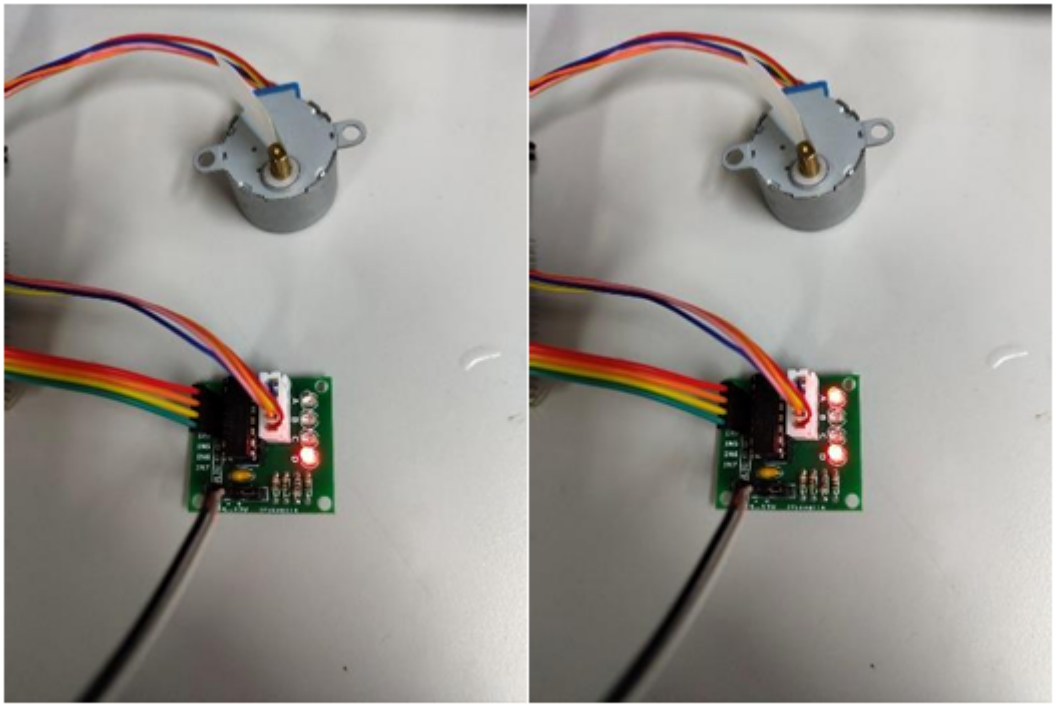


<S2>

<S3>



<S4> <S5>



<S6> <S7>

[Demo Video link]

Reference

1. Yifeng Zhu - Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C (2018)

Troubleshooting

1. When set_speed was set to (1), it did not exactly rotate 1 lap per minute. It rotate in about 58 seconds, which was faster than one minute. And when rotate 10 laps the time was 56 sec per revolution.

   => As the number of revolutions increased, errors accumulated, indicating that the clock on the stm32 board was inaccurate. To solve this problem, the value of step_delay was increased by the ratio of error.

2. Exti_interrupt, Stepper_stop did not work.

   => Since these are blocking stepper controllers, While the stepper is running, the MCU cannot process other polling commands. So I used systick_interrupt delay_ms at the same time to active Exti_interrupt.